

AI Document Assistant

Sistema RAG per l'analisi intelligente di documenti PDF

Report di Progetto

Stack: Python 3.11, FastAPI, LangChain, LangGraph
Vector DB: Qdrant
Relational: PostgreSQL 16
LLM: OpenAI GPT-4o-mini / Ollama
Deploy: Docker Compose
Versione: 1.0.0
Data: Giugno 2026

Indice

1	Introduzione	3
1.1	Scopo del Progetto	3
1.2	Caso d'Uso	3
1.3	Obiettivi Tecnici	3
1.4	Tecnologie Principali	4
2	Architettura	5
2.1	Visione d'Insieme	5
2.2	Diagramma a Blocchi	5
2.3	Flusso di Ingestion (Upload PDF)	5
2.4	Flusso RAG (Chat)	6
2.5	Motivazione per LangGraph	6
2.6	Modello Dati	7
2.6.1	PostgreSQL	7
2.6.2	Qdrant Collection	7
3	API Reference	9
3.1	Health Check	9
3.2	Endpoint Documenti	9
3.2.1	Carica un documento	9
3.2.2	Lista documenti	10
3.2.3	Dettaglio documento	10
3.2.4	Elimina documento	10
3.3	Endpoint Chat (RAG)	10
3.3.1	Fai una domanda	10
3.3.2	Storico conversazione	11
3.4	Endpoint Ricerca Semantica	12
3.5	Risultati Sperimentali	12
3.5.1	Test 1: Domanda Generale	12
3.5.2	Test 2: Architettura	13
3.5.3	Test 3: Deployment	13
4	Installazione e Avvio	14
4.1	Prerequisiti	14
4.2	Struttura del Repository	14

4.3	Configurazione	14
4.4	Avvio con Docker Compose	15
4.5	Verifica dell'Installazione	15
4.6	Comandi Make	16
4.7	Variabili d'Ambiente	16
4.8	Conclusioni e Sviluppi Futuri	17
4.8.1	Possibili Miglioramenti	17

1. Introduzione

1.1 Scopo del Progetto

L'**AI Document Assistant** è un'applicazione web full-stack progettata per consentire agli utenti di interagire in linguaggio naturale con i propri documenti PDF attraverso un sistema di *Retrieval-Augmented Generation* (RAG).

Il sistema risolve un problema pratico comune: la difficoltà di estrarre rapidamente informazioni specifiche da grandi volumi di documenti. Invece di scorrere manualmente decine di pagine, l'utente può semplicemente porre una domanda e ricevere una risposta precisa, contestualizzata e citata con i riferimenti di pagina.

1.2 Caso d'Uso

Scenario tipico: un professionista carica una raccolta di report, contratti o articoli scientifici. Vuole sapere, ad esempio:

- *"Qual è la clausola relativa alla rescissione anticipata?"*
- *"Quali metodologie statistiche sono state utilizzate nell'articolo?"*
- *"Riassumi i punti chiave del capitolo sull'architettura."*

Il sistema recupera automaticamente i passaggi più rilevanti e genera una risposta coerente, citando le pagine di origine.

1.3 Obiettivi Tecnici

1. **Caricamento e indicizzazione:** elaborazione automatica del PDF in pipeline di ingestion (parsing → chunking → embedding → storage vettoriale).
2. **Chatbot RAG:** risposta in linguaggio naturale basata esclusivamente sul contenuto del documento.
3. **Ricerca semantica:** ricerca per significato, non per keyword.
4. **Deployment containerizzato:** l'intero sistema avviabile con un singolo comando `docker compose up`.

5. **Configurabilità:** supporto per provider LLM multipli (OpenAI e Ollama per uso locale/privato).

1.4 Tecnologie Principali

Componente	Tecnologia
Backend API	FastAPI 0.115 + Uvicorn
Orchestrazione LLM	LangChain 0.3 + LangGraph 0.2
Modello linguistico	OpenAI GPT-4o-mini / Ollama Llama 3.2
Embedding	text-embedding-3-small (1536 dim) / nomic-embed-text
Vector Database	Qdrant (ricerca coseno)
Database relazionale	PostgreSQL 16 + SQLAlchemy 2.0 (async)
Parsing PDF	pypdf 5
Containerizzazione	Docker Compose

Tabella 1.1: Stack tecnologico del progetto

2. Architettura

2.1 Visione d'Insieme

Il sistema è articolato in tre macro-componenti che comunicano tra loro attraverso interfacce ben definite:

1. **Application Layer:** API REST sviluppata con FastAPI, punto di ingresso per tutti i client HTTP.
2. **Data Layer:** due database specializzati — PostgreSQL per metadati strutturati e conversazioni, Qdrant per gli embedding vettoriali.
3. **Intelligence Layer:** pipeline RAG orchestrata con LangGraph che coordina embedding, retrieval e generazione della risposta.

2.2 Diagramma a Blocchi

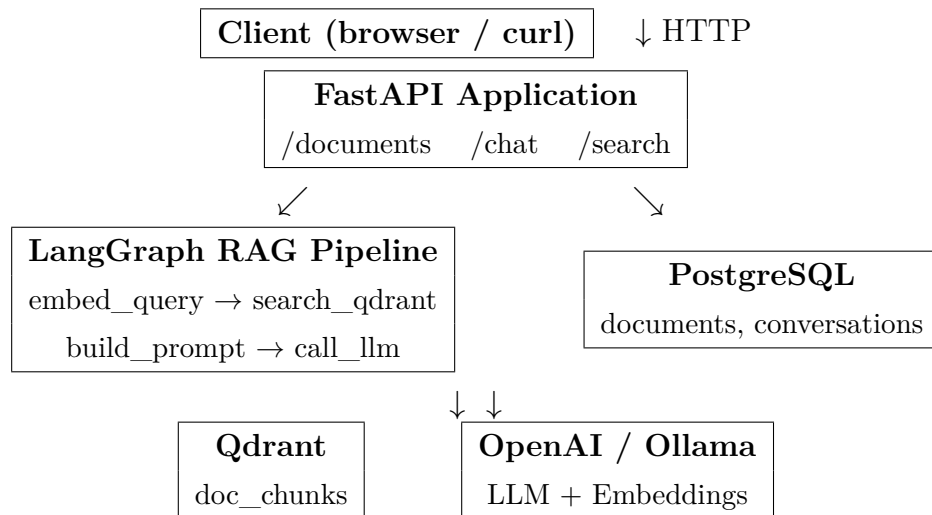


Figura 2.1: Architettura a blocchi dell'AI Document Assistant

2.3 Flusso di Ingestion (Upload PDF)

1. Il client invia il file tramite `POST /documents/upload`.
2. FastAPI valida il file (mime type, dimensione, header PDF).

3. `ingestion.py` estrae il testo pagina per pagina con **pypdf**.
4. Il testo viene suddiviso in chunk da 500 caratteri con overlap di 50 mediante `RecursiveCharacterTextSplitter` di `LangChain`.
5. Per ogni chunk viene generato un vettore di embedding (1536 dimensioni con `text-embedding-3-small`).
6. I vettori sono inseriti in Qdrant nella collection `doc_chunks` con payload: `doc_id`, `page_num`, `chunk_index`, `text`.
7. I metadati del documento (`id`, `filename`, `total_chunks`, `total_pages`, `status`) vengono salvati in PostgreSQL.

2.4 Flusso RAG (Chat)

Il flusso RAG è implementato come **grafo deterministico** con `LangGraph`. Il grafo ha quattro nodi connessi sequenzialmente:

Nodo	Funzione	Operazione
<code>embed_query</code>	Embedding	Converte la domanda in vettore
<code>search_qdrant</code>	Retrieval	Cerca i top-k chunk simili
<code>build_prompt</code>	Prompt Building	Compone il prompt con contesto
<code>call_llm</code>	Generation	Genera la risposta finale

Tabella 2.1: Nodi del grafo `LangGraph`

Lo stato condiviso tra i nodi è un dizionario tipizzato (`TypedDict`) con i campi: `question`, `doc_id`, `top_k`, `query_vector`, `retrieved_chunks`, `prompt_messages`, `answer`.

2.5 Motivazione per `LangGraph`

`LangGraph` è stato scelto per le seguenti ragioni:

- **Controllo esplicito del flusso:** ogni transizione tra nodi è dichiarata, rendendo il pipeline auditabile e modificabile senza riscrivere la logica.
- **Estendibilità:** aggiungere un nodo di reranking, di verifica delle fonti o di tool-calling richiede solo l'aggiunta di un nodo e di un edge nel grafo.
- **Compatibilità:** si integra nativamente con `LangChain`, riutilizzando `ChatOpenAI`, `OllamaEmbeddings`, ecc.

- **Streaming:** il grafo supporta lo streaming token-by-token, implementato nell'endpoint /chat con Server-Sent Events.

2.6 Modello Dati

2.6.1 PostgreSQL

```
1  -- Metadati documenti
2  CREATE TABLE documents (
3      id          VARCHAR(36) PRIMARY KEY,
4      filename    VARCHAR(512) NOT NULL,
5      original_filename VARCHAR(512) NOT NULL,
6      upload_date  TIMESTAMPTZ DEFAULT NOW(),
7      total_chunks INTEGER DEFAULT 0,
8      total_pages  INTEGER DEFAULT 0,
9      file_size_bytes INTEGER DEFAULT 0,
10     status       VARCHAR(32) DEFAULT 'processing'
11 );
12
13 -- Storico conversazioni
14 CREATE TABLE conversations (
15     id          VARCHAR(36) PRIMARY KEY,
16     session_id  VARCHAR(36) NOT NULL,
17     doc_id      VARCHAR(36) REFERENCES documents(id) ON DELETE SET NULL
18     ,
19     role        VARCHAR(16) NOT NULL, -- 'user' | 'assistant'
20     content     TEXT NOT NULL,
21     created_at  TIMESTAMPTZ DEFAULT NOW()
22 );
```

Listing 2.1: Schema tabelle PostgreSQL

2.6.2 Qdrant Collection

Ogni punto nella collection doc_chunks ha la seguente struttura:

```
1  PointStruct(
2      id=f"{doc_id}_{chunk_index}",    # ID stringa
3      vector=[0.023, -0.415, ...],     # 1536 float32
4      payload={
5          "doc_id": "uuid-string",
6          "chunk_index": 42,
7          "page_num": 3,
8          "text": "...testo del chunk..."
9      }
10 )
```

Listing 2.2: Struttura punto Qdrant

3. API Reference

La documentazione interattiva è disponibile automaticamente all'avvio del server su <http://localhost:8000/docs> (Swagger UI) e <http://localhost:8000/redoc>.

3.1 Health Check

Campo	Valore
Metodo	GET
Path	/health
Descrizione	Verifica che il server sia attivo

Risposta 200:

```
1 {"status": "ok", "version": "1.0.0"}
```

3.2 Endpoint Documenti

3.2.1 Carica un documento

Campo	Valore
Metodo	POST
Path	/documents/upload
Content-Type	multipart/form-data
Parametro	file (UploadFile, required)

Esempio richiesta:

```
1 curl -X POST http://localhost:8000/documents/upload \  
2 -F "file=@/path/to/document.pdf"
```

Risposta 201:

```
1 {  
2   "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
```

```
3  "filename": "documento_abc123.pdf",
4  "upload_date": "2026-06-11T10:30:00+00:00",
5  "total_chunks": 47,
6  "total_pages": 8,
7  "file_size_bytes": 524288,
8  "status": "ready"
9 }
```

Errori:

- 400 — File non PDF, file vuoto, o tipo MIME non valido
- 413 — Dimensione superiore al limite configurato (default 50 MB)
- 500 — Errore durante l'indicizzazione

3.2.2 Lista documenti

```
1 curl http://localhost:8000/documents/
```

Risposta 200: Array di oggetti documento, ordinati per data di upload decrescente.

3.2.3 Dettaglio documento

```
1 curl http://localhost:8000/documents/{doc_id}
```

3.2.4 Elimina documento

```
1 curl -X DELETE http://localhost:8000/documents/{doc_id}
```

Elimina il documento da PostgreSQL, i vettori da Qdrant e il file dal disco. Risposta: 204 No Content.

3.3 Endpoint Chat (RAG)

3.3.1 Fai una domanda

Campo	Valore
Metodo	POST
Path	/chat/
Content-Type	application/json

Body JSON:

```
1 {
2   "question": "Qual e la metodologia descritta nel capitolo 3?",
3   "doc_id": "3fa85f64-...", // opzionale: filtra su un documento
4   "session_id": "abc-123", // opzionale: per mantenere lo storico
5   "top_k": 4, // default: 4, range: 1-20
6   "stream": false // true per SSE streaming
7 }
```

Risposta 200 (non-streaming):

```
1 {
2   "session_id": "abc-123",
3   "answer": "Il capitolo 3 descrive una metodologia basata su...",
4   "sources": [
5     {
6       "text": "...testo del chunk rilevante...",
7       "page_num": 12,
8       "doc_id": "3fa85f64-...",
9       "chunk_index": 23,
10      "score": 0.8932
11    }
12  ]
13 }
```

Risposta streaming ("stream": true):

Server-Sent Events (SSE). Ogni evento contiene un token della risposta. L'ultimo evento è data: [DONE].

```
1 curl -X POST http://localhost:8000/chat/ \
2   -H "Content-Type: application/json" \
3   -d '{"question": "Di cosa tratta?", "stream": true}'
4 # Output:
5 # data: Il
6 # data: documento
7 # data: tratta
8 # data: di...
9 # data: [DONE]
```

3.3.2 Storico conversazione

```
1 curl http://localhost:8000/chat/history/{session_id}
```

Risposta 200: Array di messaggi con role (user/assistant) e content.

3.4 Endpoint Ricerca Semantica

Parametro	Tipo	Descrizione
q	string (required)	Testo da cercare semanticamente
top_k	integer (1-20)	Numero di risultati, default 5
doc_id	string (optional)	Filtra su un singolo documento

Esempio:

```
1 curl "http://localhost:8000/search/?q=reti+neurali&top_k=3&doc_id=3fa85f64-..."
```

Risposta 200:

```
1 {
2   "query": "reti neurali",
3   "doc_id": "3fa85f64-...",
4   "total": 3,
5   "results": [
6     {
7       "text": "Le reti neurali profonde sono composte da...",
8       "page_num": 7,
9       "doc_id": "3fa85f64-...",
10      "chunk_index": 15,
11      "score": 0.9214
12    }
13  ]
14 }
```

3.5 Risultati Sperimentali

Di seguito tre domande di test eseguite sul documento `sample.pdf` incluso nella cartella `tests/fixtures/`. Le risposte dimostrano il corretto funzionamento della pipeline RAG.

3.5.1 Test 1: Domanda Generale

Domanda: *"Di cosa tratta questo documento?"*

Risposta attesa: Il documento è un documento di esempio per testare il sistema RAG. Descrive un AI Document Assistant capace di estrarre testo da PDF, suddividerlo in chunk e indicizzare gli embedding in Qdrant.

3.5.2 Test 2: Architettura

Domanda: *"Quali sono i componenti principali dell'architettura?"*

Risposta attesa: L'architettura include FastAPI come framework web, LangChain e LangGraph per l'orchestrazione RAG, Qdrant come database vettoriale e PostgreSQL per i metadati.

3.5.3 Test 3: Deployment

Domanda: *"Come si avvia il progetto?"*

Risposta attesa: Il progetto si avvia con il comando `docker compose up`. La Swagger UI è disponibile su `http://localhost:8000/docs`.

4. Installazione e Avvio

4.1 Prerequisiti

- **Docker** $\geq$ 24.0 e **Docker Compose** $\geq$ 2.20
- **make** (opzionale, semplifica i comandi)
- **Chiave API OpenAI** (se `LLM_PROVIDER=openai`) oppure **Ollama** installato localmente (se `LLM_PROVIDER=ollama`)
- Connessione Internet per il pull delle immagini Docker al primo avvio

4.2 Struttura del Repository

```
1 ai-document-assistant/  
2 +-- app/  
3 |   +-- main.py           # Entry point FastAPI  
4 |   +-- config.py         # Impostazioni (pydantic-settings)  
5 |   +-- api/              # Router: documents, chat, search  
6 |   +-- core/             # Pipeline: ingestion, retrieval, llm  
7 |   +-- models/           # Modelli SQLAlchemy  
8 |   +-- db/               # Session, CRUD  
9 |   +-- utils/            # Validazione file  
10 +-- docker/  
11 |   +-- Dockerfile  
12 |   +-- docker-compose.yml  
13 |   +-- .env.example  
14 +-- docs/                 # Questa documentazione (LaTeX)  
15 +-- tests/  
16 |   +-- fixtures/sample.pdf  
17 |   +-- test_api.py  
18 |   +-- test_ingestion.py  
19 +-- requirements.txt  
20 +-- Makefile
```

4.3 Configurazione

1. Copiare il file di esempio:

```
1 cp docker/.env.example docker/.env
```

2. Modificare `docker/.env` con i propri valori:

```

1 # Scegliere il provider LLM
2 LLM_PROVIDER=openai          # oppure: ollama
3
4 # Obbligatorio se LLM_PROVIDER=openai
5 OPENAI_API_KEY=sk-...
6
7 # Dimensione vettori: 1536 (OpenAI) oppure 768 (nomic-embed-text)
8 QDRANT_VECTOR_SIZE=1536
9
10 # Password PostgreSQL (cambiare in produzione)
11 POSTGRES_PASSWORD=password

```

Nota per Ollama: decommentare il servizio ollama nel `docker-compose.yml` e impostare `LLM_PROVIDER=ollama`. Al primo avvio eseguire il pull del modello:

```

1 docker exec ai_doc_ollama ollama pull llama3.2
2 docker exec ai_doc_ollama ollama pull nomic-embed-text

```

4.4 Avvio con Docker Compose

```

1 # Build e avvio di tutti i servizi
2 docker compose -f docker/docker-compose.yml up --build -d
3
4 # Oppure, usando il Makefile:
5 make build
6 make up

```

I servizi avviati sono:

Servizio	Porta	URL
FastAPI app	8000	http://localhost:8000
Swagger UI	8000	http://localhost:8000/docs
PostgreSQL	5432	localhost:5432
Qdrant REST	6333	http://localhost:6333
Qdrant gRPC	6334	localhost:6334

Tabella 4.1: Porte esposte dai servizi

4.5 Verifica dell'Installazione


```

1 # Health check
2 curl http://localhost:8000/health
3 # {"status": "ok", "version": "1.0.0"}
4
5 # Carica un PDF di test
6 curl -X POST http://localhost:8000/documents/upload \
7     -F "file=@tests/fixtures/sample.pdf"
8
9 # Fai una domanda
10 curl -X POST http://localhost:8000/chat/ \
11     -H "Content-Type: application/json" \
12     -d '{"question": "Di cosa tratta il documento?"}'

```

4.6 Comandi Make

Comando	Effetto
<code>make up</code>	Avvia i container in background
<code>make down</code>	Ferma i container
<code>make build</code>	Ricostruisce le immagini
<code>make logs</code>	Mostra i log del servizio app
<code>make test</code>	Esegue pytest all'interno del container
<code>make shell</code>	Apre una shell bash nel container app
<code>make clean</code>	Rimuove container, volumi e cache Python
<code>make docs</code>	Compila la documentazione LaTeX in PDF

Tabella 4.2: Comandi disponibili nel Makefile

4.7 Variabili d'Ambiente

Variabile	Default	Descrizione
<code>LLM_PROVIDER</code>	<code>openai</code>	Provider LLM: <code>openai</code> o <code>ollama</code>
<code>OPENAI_API_KEY</code>	—	Chiave API OpenAI (obbligatoria se <code>openai</code>)
<code>OPENAI_MODEL</code>	<code>gpt-4o-mini</code>	Modello chat OpenAI
<code>OPENAI_EMBEDDING_MODEL</code>	<code>text-embedding-3-small</code>	Modello embedding OpenAI
<code>OLLAMA_BASE_URL</code>	<code>http://ollama:11434</code>	URL servizio Ollama
<code>OLLAMA_MODEL</code>	<code>llama3.2</code>	Modello chat Ollama

Variabile	Default	Descrizione
OLLAMA_EMBEDDING_MODEL	nomic-embed-text	Modello embedding Ollama
QDRANT_URL	http://qdrant:6333	URL servizio Qdrant
QDRANT_COLLECTION	doc_chunks	Nome collection Qdrant
QDRANT_VECTOR_SIZE	1536	Dimensione vettori embedding
DATABASE_URL	—	URL asyncpg PostgreSQL
CHUNK_SIZE	500	Dimensione chunk in caratteri
CHUNK_OVERLAP	50	Overlap tra chunk consecutivi
TOP_K	4	Numero chunk recuperati per query
MAX_UPLOAD_SIZE_MB	50	Limite dimensione file PDF
DEBUG	false	Modalità debug (SQL logging)

Tabella 4.3: Variabili d'ambiente configurabili

4.8 Conclusioni e Sviluppi Futuri

Il progetto raggiunge tutti gli obiettivi tecnici prefissati: un sistema RAG funzionante, containerizzato, con API documentata e supporto per provider LLM multipli.

4.8.1 Possibili Miglioramenti

- **Autenticazione:** aggiungere JWT/OAuth2 per proteggere gli endpoint.
- **Re-ranking:** aggiungere un nodo LangGraph di re-ranking (es. Cohere Rerank) per migliorare la qualità dei chunk recuperati.
- **Interfaccia Web:** sviluppare un frontend React/Next.js con chat UI.
- **Supporto multi-formato:** estendere l'ingestion a DOCX, PPTX, HTML.
- **Chunking semantico:** usare semantic chunking basato su boundary semantici invece di caratteri.
- **Cache embedding:** implementare una cache Redis per evitare il ricalcolo di embedding identici.
- **Monitoring:** integrare Prometheus/Grafana per metriche di latenza e throughput.
- **Multi-tenant:** supporto per utenti multipli con isolamento dei documenti.